

Academic Research Skills (ARS) 用户入门指南

OpenCode 学术研究技能套件·中文版

Invalid Date

目录

1	简介	2
2	安装与激活	2
2.1	前提条件	2
2.2	安装步骤	2
2.3	验证安装	3
3	快速上手	3
3.1	方式一：加载 Meta 技能（推荐起点）	3
3.2	方式二：直接使用 <code>/ars-</code> 命令	4
3.3	方式三：直接加载子技能	4
4	命令速查	4
5	五大技能详解	6
5.1	ars-meta — 智能路由	6
5.2	ars-deep-research — 深度研究（7 种模式）	6
5.3	ars-academic-paper — 学术论文（10 种模式）	7
5.4	ars-reviewer — 同行评审（6 种模式）	8
5.5	ars-pipeline — 完整管线（1 种编排模式 + 恢复模式）	9
6	管线流程概览	9
7	实用技巧	10
7.1	Token 预算参考	10
7.2	API Key 设置（可选）	10
7.3	依赖工具	10
7.4	工作流建议	10

I 简介	2
8 常见问题	10
9 附录：内容一致性校验	11

1 简介

Academic Research Skills (ARS) 是一套面向学术研究全流程的 AI 辅助技能套件，覆盖从文献检索、深度研究、论文写作、同行评审到最终发表的完整管线。无论你是刚起步的研究生，还是需要高效产出论文的科研人员，ARS 都能显著加速你的研究 workflow。

ARS 最初是为 Claude Code 开发的插件 ([imbard0202/academic-research-skills](https://github.com/imbard0202/academic-research-skills) v3.9.4.2)，现已完整移植到 **OpenCode** 平台 (版本号 3.9.4.2-oc)。

ARS 能为你做什么：

- 输入一句话研究想法，自动生成论文大纲和写作计划 (/ars-plan)
- 输入一段文字，自动搜索并匹配 Nature/CNS 系列期刊的引用文献 (/ars-lit-review)
- 上传手稿 (PDF/DOCX)，获得多角度同行评审报告 (/ars-reviewer)
- 对文稿进行 Nature 风格的英文润色、改写或中译英 (/ars-revision)
- 执行 PRISMA 系统综述流程，生成结构化综述报告 (/ars-systematic-review)
- 从论文 PDF 生成中英文对照阅读笔记 (含图表提取) (/ars-abstract)
- 一键将研究数据整合为符合 Nature 要求的数据可用性声明 (/ars-disclosure)
- 生成 Nature 风格的学术报告 PPT (中文) (/ars-plan)

2 安装与激活

2.1 前提条件

- **OpenCode** (推荐最新版本)
- Python 3.9 或更高版本
- pip (Python 包管理器)

2.2 安装步骤

第一步：克隆仓库

```
git clone https://github.com/your-org/ars-opencode.git
```

或使用本地路径：

```
~/Desktop/OpenSource/ars-opencode/
```

第二步：安装 Python 依赖（可选但推荐）

```
pip install -r requirements-dev.txt
```

部分功能（如 linter、测试工具）需要额外 Python 包。如果仅使用 LLM 驱动的技能功能，可跳过此步骤。

第三步：注册为 OpenCode 插件

如果 `ars-opencode` 尚未注册为插件，编辑 OpenCode 配置文件（`~/.config/opencode/opencode.json`），在 `plugin` 数组中添加路径：

```
{
  "plugin": [
    "...",
    "~/Desktop/OpenSource/ars-opencode"
  ]
}
```

第四步：启动新会话

关闭当前 OpenCode 会话并重新打开。新会话启动时，ARS 技能会自动加载。

2.3 验证安装

新会话启动后，输入以下命令验证 ARS 是否生效：

```
/ars-calibrate
```

如果看到 ARS 的欢迎信息或模式列表，说明安装成功。

也可以直接加载 meta 技能：

```
skill(name="ars-meta")
```

3 快速上手

以下是三种最常用的入门方式。建议按顺序体验，逐步熟悉 ARS 的能力范围。

3.1 方式一：加载 Meta 技能（推荐起点）

Meta 技能是 ARS 的路由入口。它会根据你的描述自动判断意图，将你引导到最合适的子技能。

```
skill(name="ars-meta")
```

之后只需用自然语言描述你的需求，例如：

- “帮我查一下深度学习中注意力机制的最新进展” → 自动路由到 `ars-deep-research`
- “我要写一篇关于 CRISPR 基因编辑的综述论文” → 自动路由到 `ars-academic-paper`
- “帮我 review 一下这篇 manuscript” → 自动路由到 `ars-reviewer`
- “从零开始到定稿，完整写一篇论文” → 自动路由到 `ars-pipeline`

3.2 方式二：直接使用 /ars- 命令

ARS 提供了 **13** 条预配置的快捷命令，覆盖最常用的科研场景（见表 1）。

3.3 方式三：直接加载子技能

如果你明确知道自己需要哪个技能，也可以直接加载：

```
skill(name="ars-deep-research")    # 文献检索与研究
skill(name="ars-academic-paper")   # 论文写作与润色
skill(name="ars-reviewer")         # 同行评审
skill(name="ars-pipeline")         # 完整管线编排
```

4 命令速查

ARS 共提供 **13** 条命令，按功能分为四组：研究、写作、评审、管线。

表 1: 13 条命令速查

命令	组别	用途	加载的技能	使用示例
/ars-plan	写作	通过苏格拉底式对话规划论文结构	ars-academic-paper	/ars-plan → “我要写一篇关于机器学习在药物发现中应用的综述，帮我规划结构”
/ars-abstract	写作	为论文撰写中英文摘要	ars-academic-paper	/ars-abstract → “这篇论文研究的是 CRISPR-Cas9 在肿瘤免疫治疗中的应用，帮我写中英文摘要”

命令	组别	用途	加载的技能	使用示例
/ars-revision	写作	融合审稿意见 修改手稿并生 成逐条回复	ars-academic-paper	/ars-revision → “以下是审稿人的 3 条意见和我的手稿，帮我逐条回复并修改”
/ars-format	写作	在 APA/ Chicago/ MLA/IEEE/ Vancouver 格式间转换引 用	ars-academic-paper	/ars-format → “将这篇论文的参考文献从 APA 格式转换为 IEEE 格式”
/ars-disclosure	写作	按期刊要求生 成 AI 辅助使 用声明	ars-academic-paper	/ars-disclosure → “我需要为 Nature Communications 投稿准备 AI 使用声明”
/ars-lit-review	研究	对一个主题进 行文献综述	ars-deep-research	/ars-lit-review → “帮我做一份关于联邦学习在医疗影像中应用的文献综述”
/ars-socratic	研究	通过苏格拉底 式对话引导研 究方向	ars-deep-research	/ars-socratic → “我还在纠结博士论文的方向，帮我梳理一下”
/ars-systematic-review	研究	按 PRISMA 2020 规范执 行系统综述	ars-deep-research	/ars-systematic-review → “对 AI 辅助药物发现的临床研究进行系统综述，需符合 PRISMA 规范”
/ars-fact-check	研究	逐条验证文稿 中的学术声明	ars-deep-research	/ars-fact-check → “请逐条验证这篇新闻稿中关于干细胞治疗的 5 条核心声明”
/ars-review	评审	对已有研究或 手稿进行质量 评估	ars-deep-research	/ars-review → “请评估这篇关于气候变化对生物多样性影响的研究方法和结论的可靠性”

命令	组别	用途	加载的技能	使用示例
/ars-reviewer	评审	运行完整多视角同行评审 (EIC + 3 位审稿人 + 魔鬼代言人)	ars-reviewer	/ars-reviewer → “请对这篇关于量子计算的 manuscript 进行完整的 5 视角同行评审”
/ars-calibrate	评审	用用户提供的金标准数据集校准评审器	ars-reviewer	/ars-calibrate → “以下是我标注了金标准评分的 10 篇论文，请校准评审标准”
/ars-full	管线	从研究到定稿的端到端 10 阶段管线	ars-pipeline	/ars-full → “我想从零开始写一篇关于 mRNA 疫苗递送系统的完整论文”

一致性说明: 原 DOCX 版本仅列出 9 条命令 (遗漏 /ars-socratic、/ars-systematic-review、/ars-reviewer、/ars-calibrate), 且未提供具体使用示例。本版本已补全部 13 条, 并每一条配有真实使用场景。

5 五大技能详解

ARS 包含 5 个核心技能 (skills), 每个技能下辖若干工作模式 (modes), 总共 25 种模式。

5.1 ars-meta — 智能路由

定位: 自动分析用户意图, 将请求路由到正确的子技能。推荐作为所有用户的起点。

触发方式:

```
skill(name="ars-meta")
```

典型输入: - “research topic” / “literature review” → 路由到 ars-deep-research - “write paper” / “draft manuscript” → 路由到 ars-academic-paper - “review paper” / “peer review” → 路由到 ars-reviewer - “full pipeline” / “complete paper” → 路由到 ars-pipeline

5.2 ars-deep-research — 深度研究 (7 种模式)

定位: 从快速文献检索到 PRISMA 系统综述的全谱研究需求。整合了 PubMed、CrossRef、arXiv 等学术搜索引擎。

模式	中文名	说明	适用场景
full	完整研究	多智能体团队, 3k-8k 字 APA 报告	需要全面的研究报告
quick	快速简报	500-1.5k 字研究摘要	快速了解一个方向
lit-review	文献综述	注释文献列表 + 综合评述	写综述前的文献梳理
systematic-review	系统综述	5k-15k 字 PRISMA 报告	系统综述 / Meta 分析
fact-check	事实核查	逐条声明的证据等级验证	验证学术声明
review	质量审查	研究成果的质量评估报告	评估已有文献或成果
socratic	苏格拉底引导	通过对话引导深入研究方向	研究方向不确定时

5.3 ars-academic-paper — 学术论文 (10 种模式)

定位：从论文规划到发表的全程支持。支持 IMRaD 结构写作、中英文摘要、多格式引用转换 (APA/Chicago/MLA/IEEE/Vancouver) 等。

模式	中文名	说明	适用场景
full	完整草稿	生成 IMRaD 完整论文草稿	从零写论文
plan	规划引导	苏格拉底式对话引导论文框架	需要论文结构规划
outline-only	仅提纲	详细大纲 + 证据图谱	只要大纲不要正文
revision	修改草稿	融合审稿意见的修订 + 逐条回复	修改手稿 (有审稿意见)
revision-coach	修改指导	审稿意见解析 + 回复策略建议	不知道怎么修改
abstract-only	摘要生成	中英文双语摘要 + 关键词	写摘要
lit-review	综述论文	按论文格式撰写文献综述	写综述论文

模式	中文名	说明	适用场景
format-convert	格式转换	LaTeX/DOCX/ PDF/Markdown 互转	格式转换需求
citation-check	引用核查	引用格式和完整性检查	检查参考文献
disclosure	AI 声明	按期刊要求生成 AI 辅助使用声明	投稿前准备

5.4 ars-reviewer — 同行评审（6 种模式）

定位：多角度（方法学/统计学/伦理学/再现性/写作质量）同行评审，含 0-100 结构化评分。

模式	中文名	说明	适用场景
full	完整评审	5 份评审报告 + 编辑 决定 + 修改路线图	对手稿做完整评审
re-review	重新评审	修订核查清单 + 遗 留问题标识	核查修订是否到位
quick	快速评审	主编级快速评估 + 关键问题列表	快速检查硬伤
methodology-focus	方法学专项	深度的实验设计和方 法学评审	重点关注方法学
guided	引导式评审	逐问题苏格拉底式对 话改进	逐轮改进论文
calibration	校准模式	评审准确度测量 (FNR/FPR/AUC)	校准评审标准

评分标准：

分数	决定
≥ 80	Accept (接受)
65-79	Minor Revision (小修)
50-64	Major Revision (大修)
< 50	Reject (拒稿)

5.5 ars-pipeline — 完整管线 (1 种编排模式 + 恢复模式)

定位：将前 4 个技能编排为 10 阶段端到端管线：Research → Write → Integrity Check → Review → Revise → Re-review → Final Integrity → Finalize → Summary。每个阶段结束后有质量门禁和检查点。

典型 Token 消耗：完整管线约 \$4-6 (15,000 词论文)，单个技能 \$1-3。

6 管线流程概览

ARS 的核心特色之一是完整的 10 阶段学术发表管线，将”研究 → 写作 → 评审 → 修改 → 定稿”的完整流程串接起来。

阶段	说明	使用技能	用户决策
1. RESEARCH	深度研究，确定 RQ 和方法	deep-research	确认 RQ Brief + Methodology Blueprint
2. WRITE	论文写作，生成 IMRaD 草稿	academic-paper	确认大纲
2.5 INTEGRITY	7 维完整性自动检查	自动化	确认 Integrity Report
3. REVIEW	多角度同行评审	reviewer	选择 Accept / Minor / Major / Reject
3→4 Coaching	修改策略引导（最多 8 轮 Socratic 回合）	academic-paper	确认修改策略
4. REVISE	根据评审意见修改	academic-paper	确认修改内容
3'. RE-REVIEW	复核修改是否到位	reviewer	确认复核结果
4.5 FINAL INTEGRITY	最终完整性检查 (Mode 2 深度检测)	自动化	确认 Final Integrity Report
5. FINALIZE	选择输出格式 (MD/DOCX/LaTeX/PDF)	academic-paper	选择格式
6. SUMMARY	流程总结 + 质量评审	pipeline	确认语言 + 协作质量

每个阶段结束后都有用户确认检查点。设计理念：AI 效率 + 研究者自主权。

7 实用技巧

7.1 Token 预算参考

任务类型	预估 Token	预估费用 (USD)
完整管线 (15k 词论文)	~150k-250k	\$4-6
单个技能 (深度研究)	~50k-100k	\$1-3
快速任务 (文献综述)	~20k-50k	\$0.5-1
简单任务 (格式转换)	~5k-15k	\$0.1-0.3

7.2 API Key 设置 (可选)

部分功能使用学术搜索引擎 API，设置 Key 可以获得更高速率限制：

```
export S2_API_KEY=your_semantic_scholar_key
export CROSSREF_MAILTO=your@email.com
```

7.3 依赖工具

- **Pandoc** (可选)：用于 DOCX/LaTeX 格式输出
- **tectonic + Source Han Serif TC** (可选)：用于 PDF 输出
- **Python 3.9+**：运行测试和 lint 脚本

7.4 工作流建议

- 新手建议从 /ars-plan 开始，先体验论文规划流程
- 熟悉后再尝试 /ars-lit-review 进行快速文献检索
- 有完整论文需求时使用 /ars-full 管线
- 定期运行以下命令检查功能完整性：

```
python3 -m pytest scripts/ -v
```

8 常见问题

Q: 安装后技能没有出现怎么办？

确保 opencode.json 中正确配置了 plugin 路径，然后启动全新的 OpenCode 会话（关闭当前会话后重新打开）。ARS 插件在当前会话中不会生效，需要在新会话中自动加载。

Q: 命令 /ars-xxx 不识别怎么办?

检查 `opencode.json` 的 `commands` 部分是否正确配置了 13 条命令的路径。确认路径相对于仓库根目录正确。

Q: 运行测试报错怎么办?

确保已安装 `requirements-dev.txt` 中的依赖。部分测试需要网络访问权限。已知约 100 个测试用例需要上游环境支持，不影响核心功能使用。

Q: ARS 和 Claude Code 版本有什么区别?

核心内容 (Agent prompts、脚本、schemas、文档) 100% 同步上游 v3.9.4.2。差异仅在于调度层：CC 使用 `Agent tool + model` 字段，OC 使用 `task() + category` 参数。参考项目根目录的 `SYNC.md` 了解同步机制。

Q: ARS 需要联网吗?

大多数功能需要联网（文献检索、引用匹配等）。论文写作、润色、格式转换等可离线操作，但需本地 LLM 支持。

9 附录：内容一致性校验

本文档在发布前已通过以下自动校验：

1. 命令数量校验： `opencode.json` 中定义的 13 条命令，在上文表 1 中全部列出 (13/13 条)，无遗漏
2. 技能数量校验： `opencode.json` 中定义的 5 个技能，在第 5 章逐一介绍 (5/5 个)，无遗漏
3. 模式数量校验： `MODE_REGISTRY.md` 中的 25 种模式，在第 5 章按技能分类列出 (7+10+6+1+1=25)，无遗漏
4. 示例完整性： 13 条命令每一条都配有真实使用场景和示例输入，无空白条目
5. 数值一致性： 全文“13 条命令”、“5 个技能”、“25 种模式”等关键数字均与实际内容匹配